

# IDENTITY IS THE SIGNAL: WHY PRE-EXECUTION AGENT-ACTION RISK NEEDS DELEGATED AUTHORITY, NOT JUST CONTENT

Anonymous authors

Paper under double-blind review

## ABSTRACT

Guardrail models for AI agents typically classify a proposed action from its text alone. We show this misses the dimensions that matter most for delegated, tool-using agents. Stripping principal entitlements, delegated scopes, and policy text from a trained 2B-parameter decoder’s input drops macro AUPRC from 0.915 to 0.736 on held-out tools, concentrated entirely in three dimensions — unauthorized-scope detection falls from 1.00 to 0.12, policy-conflict detection from 1.00 to 0.35 — while two zero-shot safety models (Granite Guardian 3.3 8B, gpt-oss-safeguard 20B) sit at chance on exactly these dimensions with or without that context, showing the gap is not something prompting a general safety model can close. We further show that the *shape* of policy-conditioned reasoning, not just tool identity, must generalize to unseen policy kinds and phrasings, and that naive multi-policy composition collapses to a 0.97 false positive rate by three stacked policies unless decisions are made from calibrated joint probabilities rather than independent thresholds. An independent blind LLM judge, shown only the rendered text, confirms two of the three dimensions the identity-ablation result rests on (policy-conflict, unauthorized-scope) and finds the third (insufficient-context) unrecoverable from the text at all, which we report as an open validity gap. On AgentDojo banking traces we did not generate, zero-shot, the 2B checker reaches 0.85 AUPRC on policy conflict against a 0.24 base rate and the 8B checker 0.70 AUPRC on prompt-injection influence against a 0.15 base rate, both against rule baselines at or near base rate, while the 2B checker fails outright on injection. On InjecAgent, which holds the call fixed and varies only whether the attacker’s instruction is in the tool output, neither v4 checker separates poisoned from clean (AUROC 0.53–0.60), so what transferred was call-level judgement, not reading of untrusted text. We trace this to a label leak in our generator, fix it, and retrain: the leak-free checkers reach 0.69 (2B) and 0.84 (8B) AUPRC on AgentDojo injection, both rank the poisoned InjecAgent variant above its clean twin on 98% of pairs (50% and 35% before), and the synthetic injection score does not move (0.80 to 0.79), so the leak had hidden what was learned, not inflated the score. All other results are on synthetic data; we are explicit about what that does and does not license.

## 1 INTRODUCTION

Deployed AI agents increasingly act with delegated authority: a support agent issues refunds for a merchant, a coding agent pushes commits for a developer, an operations agent grants IAM roles for an administrator. Before such an action executes, some system must decide whether it is safe to proceed. The dominant tool is a guardrail classifier that reads the proposed action and outputs a harm judgment (Inan et al., 2023; Padhi et al., 2024; Zeng et al., 2024; Ghosh et al., 2024; OpenAI, 2025).

We argue this conflates two questions. **Content risk** asks whether an action looks dangerous on its face (deletes data, moves money, contacts an external party) and is answerable from the action text alone. **Authorization risk** asks whether *this* action is something *this* agent was allowed to do, on behalf of *this* principal, under the policy in force – and requires the principal’s entitlements, the

agent’s delegated scope, and the applicable policy clause as structured inputs, not prose the model must infer.

We build a pre-execution risk classifier, *forecheck*, whose input contract makes these fields explicit, then strip them from the same trained model’s input to see what breaks. On synthetic data, three of eleven risk dimensions collapse from near-perfect to near-chance when identity is removed, while dimensions already recoverable from action text alone are unaffected. This is not a fine-tuning artifact prompting could fix: two zero-shot safety models given the same policy text in-prompt are at chance on exactly these dimensions, with or without identity, because they were never trained to use it. We further test whether policy-conditioned reasoning generalizes to unseen policy *kinds* and *phrasings*, and show that composing several such classifiers’ decisions – how guardrail stacks are deployed in practice – fails in a specific, fixable way: per-policy thresholding compounds toward denying everything as more policies are added, while deciding from the joint calibrated probability vector does not. On AgentDojo banking traces we did not generate, zero-shot, the 2B checker reaches 0.85 AUPRC on policy conflict against a 0.24 base rate and the 8B checker 0.70 AUPRC on prompt-injection influence against a 0.15 base rate, both against rule baselines at or near base rate, while the 2B checker fails outright on injection until a label leak in our generator is fixed, after which it reaches 0.69 and the 8B checker 0.84, and both then separate poisoned from clean InjecAgent tool outputs on 98% of pairs. On InjecAgent, where the call is held fixed and only the tool output changes, neither v4 checker reads the injected instruction (pair accuracy at or below chance); the policy result transfers a second time (0.98 AUPRC against 0.46).

**Contributions.** (1) An identity-ablation diagnostic separating which of eleven risk dimensions need delegated-authority context from which are content lookups (§3); (2) evidence that two current zero-shot safety models do not use delegated-authority context even when it is handed to them (§3); (3) a policy-generalization result showing robustness to unseen policy wording but not unseen policy kinds (§4); (4) a multi-policy composition result showing calibrated joint-probability decisions, not rule engineering, keep a guardrail stack from collapsing to "deny everything" (§5); (5) a comparison showing an agent asked to judge its own action cannot recover the authorization signal an external checker can (§6); and (6) an independent blind-judge check of whether the generator’s labels are recoverable from the rendered text at all, which confirms the identity-ablation result on two of its three load-bearing dimensions and flags the third as unvalidated (§7); (7) zero-shot transfer results on AgentDojo and InjecAgent, two published external benchmarks, showing that policy-conflict transfers to agent traces and policies *forecheck* never saw, that the v4 injection result is call-level rather than a reading of untrusted text and the leak-free v6 result is not, that unauthorized-scope does not transfer, and that calibration needs a few hundred in-domain labels (§8); (8) a label-leak diagnosis of our own generator that invalidates every synthetic prompt-injection number before v6, and the leak-free retrain that turns the 2B injection failure on AgentDojo into a transfer (§10, §8). Every number here except §8 is on synthetic data from *forecheck*’s own scenario generator; §10 states plainly what that does and does not license.

## 2 PROBLEM SETTING

**Contract.** *forecheck* scores a proposed tool call against an *ActionContext*: the principal’s entitlements, the scope actually delegated to the agent (a possibly strict subset of the principal’s), a trust-labelled trajectory of prior observations, the applicable policy clause(s), and the proposed action (tool, arguments, destination) – typed fields the classifier reads directly, not facts inferred from prose. A classifier that sees only the rendered action text, as every guardrail model in §9 does, can answer content-risk questions but has no way to check whether the agent was delegated the scope it is exercising.

**Eleven risk dimensions.** Four are *content/consequence* dimensions decidable from the action and destination alone: *destructive\_or\_irreversible\_action*, *financial\_commitment*, *external\_communication*, *untrusted\_destination*. Two are *trajectory* dimensions decidable from prior observations without identity: *prompt\_injection\_influence*, *suspicious\_action\_sequence*. The remaining are where identity and policy are load-bearing: *unauthorized\_scope* (inside the principal’s entitlements *and* the agent’s delegated scope?), *policy\_conflict* (violates a stated clause?), *insufficient\_context* (enough delegated information to decide at all?),

and `privilege_escalation` (does the action widen authority beyond what was granted?). `sensitive_data_exposure` behaves like a content dimension in practice (§3) because its deciding field renders directly into the action arguments.

**Data.** All results are on synthetic data from forecheck’s own scenario generator (49,996 rows at the v2/v3/v4 scale used here), split into `train/calibration/ dev/test`, a `heldout_family` split withholding a stratified subset of tools, and, from v3 on, `heldout_policy_kind/heldout_policy_phrasing` splits (§4). Every number measures whether an arm learned the generator’s label function, not real-world risk; no human-labelled or production-traffic evaluation exists at submission time (§10).

**Arms.** A LoRA-adapted 2B decoder (`granite-3.3-2b-instruct`) scored by candidate-token logits over the eleven dimensions (Hu et al., 2021); two linear-head encoders (ModernBERT-large (Warner et al., 2024), `granite-embedding-r2`); a deterministic rule baseline with no learned component; two zero-shot safety models with the same policy text in-prompt and no fine-tuning, Granite Guardian 3.3 8B (Padhi et al., 2024) and `gpt-oss-safeguard` 20B (OpenAI, 2025); and an *agent self-judgment* condition where an untrained instruct model (2B, 8B (IBM Granite Team, 2024)) gets the same context in a Fin-style system prompt and returns one ALLOW/STOP verdict (§6).

### 3 IDENTITY ABLATION

At evaluation time only, we remove principal entitlements, delegated scope, `on_behalf_of`, and policy text from the trained decoder’s rendered context (`-strip-identity`), leaving action, destination, and trajectory untouched, and compare against a rule baseline (action and destination fields only) and two zero-shot safety models given the same policy text in-prompt.

Table 1: identity-aware trained models lose 0.10–0.18 macro AUPRC when identity is removed; the rule baseline is unchanged; the zero-shot models are flat or slightly *improve*. If a general safety model quietly used delegated-authority context, stripping it would hurt – it does not, for either model.

The v4 ablation replicates the v2 pattern on the fixed data: macro AUPRC drops 0.960 to 0.790, close to v2’s 0.915 to 0.736. The one change is `privilege_escalation`, which now survives stripping entirely (1.000 to 1.000): the v4 render fix makes `authority_before/authority_after` observable directly in the tool-call text, so it is a content dimension, not an identity one. An 8B decoder on the same v4 data shows the identical collapse (0.958 to 0.786, Table 1), with the same three dimensions falling and the same two surviving – model scale gives the classifier no way to infer entitlements or policy text that were simply not in its input.

Training on data rendered without identity fields from the start, rather than stripping only at evaluation time, gives the same ceiling: macro AUPRC 0.780/0.784 on test/heldout\_family, within a

Table 1: Macro AUPRC with and without identity/policy context. Decoder and encoder rows are on `heldout_family` unless noted; zero-shot rows use a 2,000-row (Granite Guardian) or 400-row (`gpt-oss-safeguard`) subsample. Zero-shot models are unaffected or slightly helped by stripping, showing they never used the context. Source: `docs/results/synthetic-v2/README.md`.

| Arm                                            | Split                                | Macro AUPRC (full) | Macro AUPRC (stripped) |
|------------------------------------------------|--------------------------------------|--------------------|------------------------|
| Decoder 2B v2                                  | <code>heldout_family</code>          | 0.915              | 0.736                  |
| Decoder 2B v4                                  | <code>heldout_family</code>          | 0.960              | 0.790                  |
| Decoder 8B v4                                  | <code>heldout_family</code>          | 0.958              | 0.786                  |
| Decoder 2B v4, trained stripped                | <code>heldout_family</code>          | —                  | 0.784                  |
| Encoder ModernBERT-large                       | <code>heldout_family</code>          | 0.813              | 0.712                  |
| Encoder <code>granite-embedding-r2</code>      | <code>heldout_family</code>          | 0.841              | —                      |
| Granite Guardian 3.3 8B (zero-shot)            | <code>heldout_family</code>          | 0.257              | 0.271                  |
| <code>gpt-oss-safeguard</code> 20B (zero-shot) | test (n=400)                         | 0.269              | 0.238                  |
| Agent self-judgment 8B <sup>a</sup>            | <code>heldout_family</code> (n=2000) | 0.242              | 0.281                  |
| Rule baseline                                  | <code>heldout_family</code>          | 0.614              | 0.614                  |

<sup>a</sup>Single ALLOW/STOP verdict scored against all eleven labels (§6), not a per-dimension classifier.

Table 2: Decoder 2B v2, heldout\_family AUPRC per dimension: full, identity-stripped, and rule baseline. First three rows are delegated-authority dimensions; the last is a representative content/lookup dimension. Source: docs/results/synthetic-v2/README.md.

| Dimension                     | Decoder (full) | Decoder (stripped) | Rule baseline |
|-------------------------------|----------------|--------------------|---------------|
| unauthorized_scope            | 1.000          | 0.116              | 0.512         |
| policy_conflict               | 0.999          | 0.345              | 0.315         |
| insufficient_context          | 0.849          | 0.418              | 0.235         |
| financial_commitment (lookup) | 1.000          | 1.000              | 1.000         |

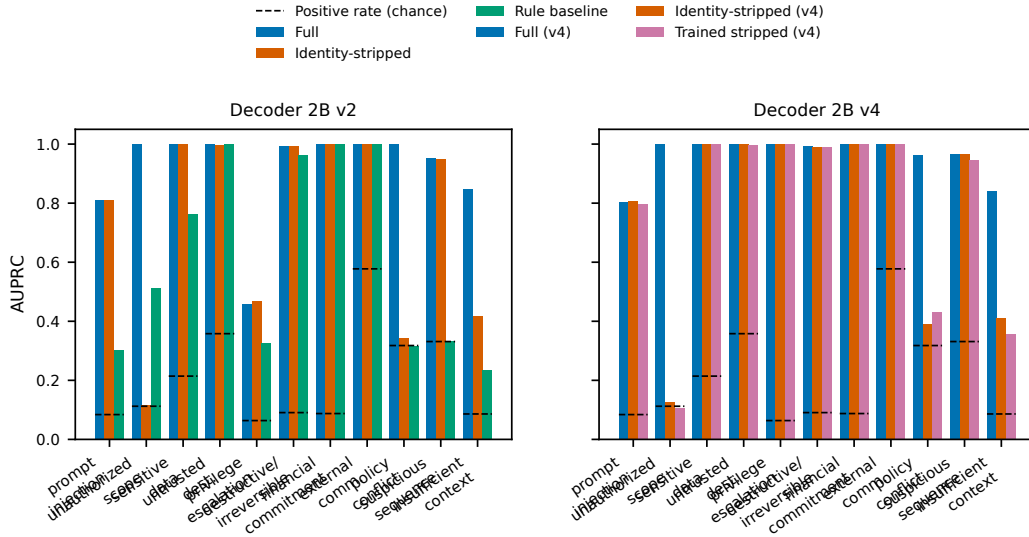


Figure 1: Decoder 2B v2, per-dimension AUPRC on heldout\_family ( $n=6640$ , synthetic data): full model vs. identity-stripped vs. rule baseline, with each dimension’s positive rate (chance level) marked by a dashed line. Source: docs/results/synthetic-v2/decoder-2b/.

point of the eval-time-stripped model’s 0.787/0.790. The three delegated-authority dimensions still do not recover: heldout\_family unauthorized\_scope falls to 0.107, policy\_conflict to 0.429, insufficient\_context to 0.357, matching or below the eval-time-stripped model and nowhere near the full model’s 0.999/0.964/0.841. This rules out the reading that eval-time stripping merely broke a fragile mechanism the model had learned to use – the information is absent from the rendered content itself.

Figure 1 shows the same pattern across all eleven dimensions: the drop from stripping identity is concentrated in unauthorized\_scope, policy\_conflict, and insufficient\_context, while the remaining dimensions barely move.

The macro drop is concentrated entirely in the three delegated-authority dimensions. unauthorized\_scope falls below the rule baseline (0.116 vs. 0.512) once the entitlements it must be checked against are gone; policy\_conflict falls to 0.345, barely above the 0.315 rule floor, because there is no clause left to violate; insufficient\_context falls by half. The fourth row is untouched: its deciding fact renders directly into the action arguments regardless of identity, and the rule baseline already gets it right. Four more dimensions (destructive\_or\_irreversible\_action, untrusted\_destination, external\_communication, sensitive\_data\_exposure) show the same flat, lookup pattern; two more (prompt\_injection\_influence 0.810, suspicious\_action\_sequence 0.952) are learned from trajectory order and are identity-independent.

The zero-shot models’ near-chance macro AUPRC (0.24–0.27), unmoved by identity, shows this is not a prompting gap: both were trained for harm, jailbreak, and groundedness criteria, not for checking an action against entitlements or policy. gpt-oss-safeguard is also  $\sim 50\times$  slower per row than the 2B decoder, for no benefit on the dimensions that matter here.

This whole claim rests on the three delegated-authority dimensions being real labels, not generator artifacts an independent reader cannot recover; §7 checks that directly and finds the claim solid for two of the three, with `insufficient_context` unvalidated.

#### 4 POLICY GENERALIZATION: KIND VS. PHRASING

Robustness to unseen *tools* (`heldout_family`) is not robustness to unseen *policy kinds* or *phrasings*, and the latter is the more product-relevant test: a deployment needs the classifier to apply a customer’s own wording, not just handle a new tool under a training-set policy. ADR 0010 adds two splits: `heldout_policy_kind` withholds entire predicate kinds (e.g. `forbid_recipient_domain`) from training; `heldout_policy_phrasing` withholds one paraphrase index of every kind the model *did* train on. v4 additionally fixes a renderer defect that left `privilege_escalation` undecidable from text (§10).

Figure 2 makes the kind-vs-phrasing asymmetry visible: bars for test, `heldout_family`, and `heldout_policy_phrasing` sit close together near the ceiling, while `heldout_policy_kind` drops toward, though still above, the rule baseline line, for every generator and both model sizes.

**Unseen phrasing generalizes.** A paraphrase index never seen in training leaves `policy_conflict` at 0.980–0.993 AUPRC, matching the in-distribution test split (0.990–0.996): the model reads the clause’s meaning, not a template id. **Unseen policy kinds do not, even with twice as many kinds.** ADR 0011’s v5 triples the kind inventory (15 to 30) while withholding the same proportion (4 of 30 vs. 2 of 15). Across three training seeds, `policy_conflict` on `heldout_policy_kind` lands at 0.704, 0.755, and 0.717 AUPRC (mean 0.725, sample sd 0.026,  $n=3$ ) – essentially unchanged from the v4 15-kind recipe’s own three-seed band of 0.752, 0.689, and 0.711 (mean 0.717, sample sd 0.032,  $n=3$ ; §10). The v5 mean sits within one sample sd of the v4 mean, so tripling the kind inventory is not a real improvement, it is inside the seed noise the 15-kind recipe already showed. The rule baseline on this split is 0.325, so the model is still far above chance; it has just not learned the general operation. **Model size helps, partially.** An 8B LoRA on the unchanged v4 data reaches 0.851 AUPRC (AUROC 0.916) on the same `heldout_policy_kind` `policy_conflict` cell – a real gain over both 2B bands – but still well short of the  $\sim 0.98$  phrasing-generalization ceiling both sizes reach. Capacity recovers some of the missing generalization; it does not close the gap.

The v4 fix also confirms `privilege_escalation`’s earlier AUROC-high/AUPRC-low signature (§10) was a data defect, not a model limit: rendering the deciding `authority_before/authority_after` values lifts it from 0.505 to 1.000 AUPRC on `heldout_family`.

Table 3: Decoder 2B, macro AUPRC and `policy_conflict` (pc) AUPRC across splits, on the v3 (ADR 0010, 15 policy kinds, 2 withheld), v4 (same data, `privilege_escalation` render fix), and v5 (ADR 0011, 30 kinds, 4 withheld) generators. Last column: an 8B LoRA on the unchanged v4 data, `policy_conflict` only. \*v5 `heldout_policy_kind` pc is seed 0 of three training seeds (mean 0.725, discussed in text). Source: docs/results/synthetic-v2/README.md.

| Split                                | v3 macro | v4 macro | v5 macro | v3 pc        | v4 pc        | v5 pc         | 8B v4 pc     |
|--------------------------------------|----------|----------|----------|--------------|--------------|---------------|--------------|
| test                                 | 0.908    | 0.960    | 0.957    | 0.994        | 0.994        | 0.996         | 0.990        |
| <code>heldout_family</code>          | 0.914    | 0.960    | 0.951    | 0.941        | 0.964        | 0.933         | 0.965        |
| <code>heldout_policy_kind</code>     | 0.889    | 0.944    | 0.941    | <b>0.653</b> | <b>0.752</b> | <b>0.704*</b> | <b>0.851</b> |
| <code>heldout_policy_phrasing</code> | 0.921    | 0.968    | 0.970    | 0.987        | 0.993        | 0.980         | 0.978        |

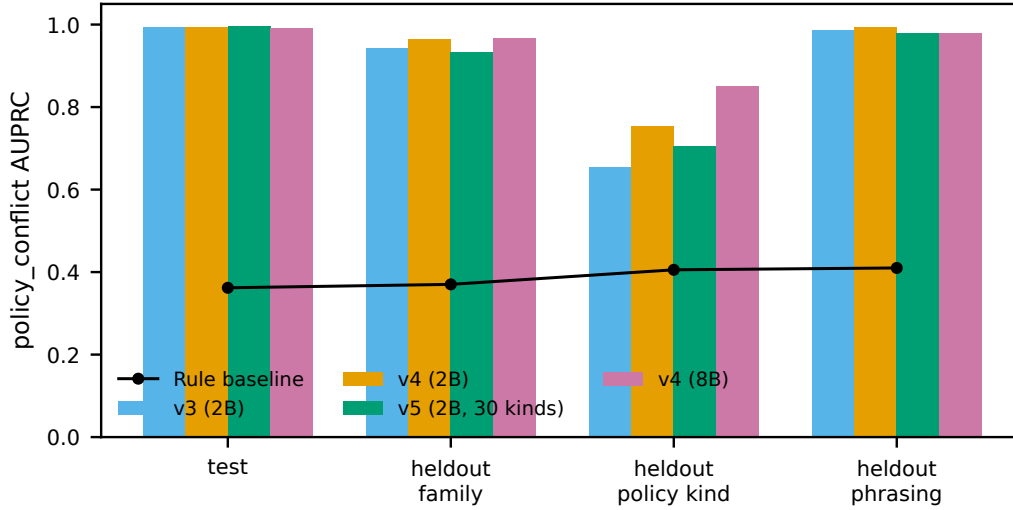


Figure 2: Decoder 2B, `policy_conflict` AUPRC across splits for the v3, v4, and v5 generators, plus the 8B LoRA on v4 data (synthetic data;  $n$  per split matches Table 3), with the rule baseline’s `policy_conflict` AUPRC on each split shown as a line. Source: docs/results/synthetic-v2/decoder-2b-v3/, decoder-2b-v4/, decoder-2b-v5/, decoder-8b-v4/.

## 5 MULTI-POLICY COMPOSITION AND THE CALIBRATION HINGE

Guardrail deployments commonly run several independently-tuned policies at once and act on the most restrictive decision – OR-ing  $k$  binary guards. Each guard’s false-positive rate compounds under that OR purely as a function of  $k$ , independent of any guard getting worse. forecheck’s alternative is one calibrated probability vector over the eleven dimensions, composed by one policy engine, decided either by per-dimension thresholds (`threshold` mode) or by the argmin of expected cost over the joint vector (`expected_cost` mode).

Figure 3 shows the independent and joint threshold strategies’ FPR climbing sharply as  $k$  grows past 2, while `expected_cost_joint` stays flat; the encoder’s `expected_cost_joint` curve, by contrast, sits near the top of the FPR panel throughout, illustrating that the flat curve is a property of the decoder’s calibration, not of the decision rule alone.

Threshold-based independent stacking pushes the decoder’s FPR from 0.015 at  $k=1$  to 0.966 by  $k=3$ , staying there through  $k=8$  – effectively denying everything, not because any policy got worse but purely because more were added. Deciding from the joint calibrated probability vector under `expected_cost` is invariant to  $k$  from  $k=2$  on (FPR/FNR/review rate flat at 0.082/0.070/0.28): the decision depends on the probability vector and a fixed cost table, not on how many rules mention a dimension, so stacking more policies cannot by itself make the classifier more conservative.

Table 4: Decoder 2B, false positive rate on covered-benign examples, `heldout_family` ( $n=6640$ ), stacking  $k$  synthetic per-dimension policies. `independent` ORs  $k$  separately-thresholded decisions; `expected_cost_joint` merges the same  $k$  policies into one bundle and decides from the joint probability vector. Full table with the `joint` strategy, FNR, review rate, and the granite-embedding-r2 encoder is Table 10 in the appendix. Source: docs/evaluation/multi-policy-stacking.md.

| $k$                     | 1     | 2     | 3 (8 identical) |
|-------------------------|-------|-------|-----------------|
| independent FPR         | 0.015 | 0.140 | <b>0.966</b>    |
| expected_cost_joint FPR | 0.015 | 0.082 | 0.082           |

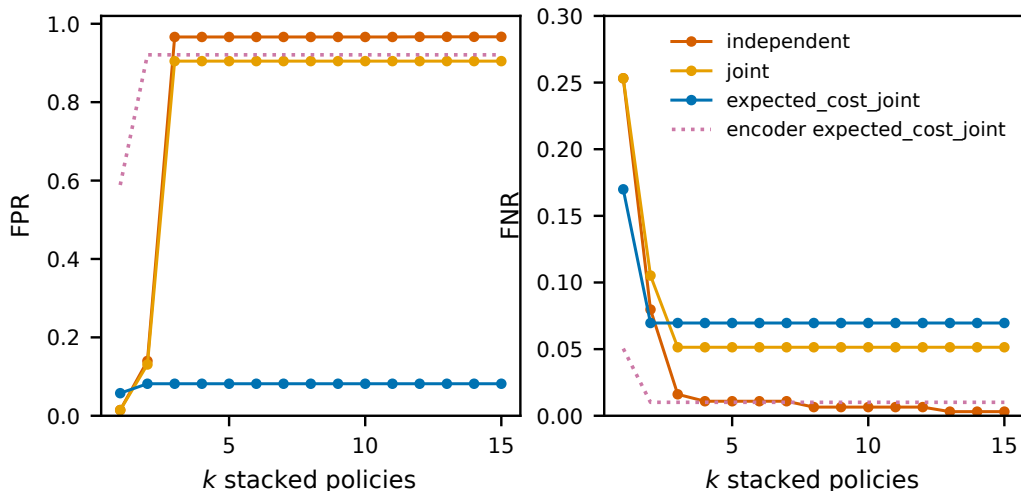


Figure 3: Decoder 2B, FPR (left) and FNR (right) vs. number of stacked policies  $k$  on heldout\_family ( $n=6640$ , synthetic data), for independent, joint, and expected\_cost\_joint strategies; the granite-embedding-r2 encoder’s expected\_cost\_joint curve is shown dotted for comparison. Source: docs/results/synthetic-v2/decoder-2b/heldout\_family-stacking-report.md, encoder-granite-embedding-r2/heldout\_family-stacking-report.md.

**The invariance is a calibration property, not a decision-rule property.** The same rule on the granite-embedding-r2 encoder’s probabilities – identical examples and policies – settles at FPR 0.921, not 0.08 (Table 10). The decoder is well calibrated (macro ECE 0.004 on heldout\_family, temperature-scaled on the calibration split (Guo et al., 2017; Platt, 1999)); the encoder is flatter (ECE 0.049). Expected-cost composition only keeps a stack from collapsing when the probabilities feeding it are sharp and calibrated, and cannot be recovered by rule engineering alone on a poorly-calibrated model.

The cost table used throughout is the default severity-tier table, not tuned to any deployment: the  $k=1$  row already trades false positives for false negatives relative to the F1-optimal threshold (FPR/FNR 0.058/0.170 vs. 0.015/0.253). The  $k$ -invariance claim is about the *shape* of the composition curve, not that this default cost table is right for any given customer.

## 6 CAN THE ACTING AGENT JUDGE ITS OWN SCOPE?

The classifier above is an external checker: it never generates the action, only scores it. A cheaper alternative is to ask the acting agent itself, from inside its own system prompt, whether to proceed. We give the same untrained instruct models used as the decoder’s base (2B, 8B) a Fin-style system prompt carrying the identical entitlements, delegated scope, and policy text, followed by the pending tool call, and read a single ALLOW/STOP verdict off the first output token. That one scalar is scored against all eleven dimensions’ labels: tracking the external checker means an undifferentiated act/refuse decision happens to catch that risk; lagging badly means the dimension needs external, per-dimension structure the agent’s own judgment does not supply.

Figure 4 shows the pattern holds across all eleven dimensions, not just the five in Table 5: the agent’s bars track the chance line closely everywhere except policy\_conflict and external\_communication, while the external checker clears 0.84 AUPRC or better on every dimension.

Two findings. First, **the agent cannot judge its own scope from inside its prompt**: with full identity context the 8B agent is at or near chance on every identity-dependent dimension (AUROC 0.53–0.67, not shown), the same level as the zero-shot safety models in §3. The delegation

Table 5: Agent self-judgment (8B, untrained, single ALLOW/STOP verdict) vs. the trained decoder external checker (v4), AUPRC on heldout\_family ( $n=2000$  for the agent rows). Positive rate for each dimension in parentheses. Source: docs/results/synthetic-v2/README.md.

| Dimension (pos. rate)              | Agent 8B | Agent 8B, stripped | Decoder 2B v4 (checker) |
|------------------------------------|----------|--------------------|-------------------------|
| unauthorized_scope (0.106)         | 0.138    | 0.104              | 0.999                   |
| policy_conflict (0.385)            | 0.509    | 0.415              | 0.964                   |
| insufficient_context (0.079)       | 0.085    | 0.084              | 0.841                   |
| privilege_escalation (0.075)       | 0.103    | 0.106              | 1.000                   |
| prompt_injection_influence (0.075) | 0.169    | <b>0.498</b>       | 0.804                   |
| Macro                              | 0.242    | 0.281              | 0.960                   |

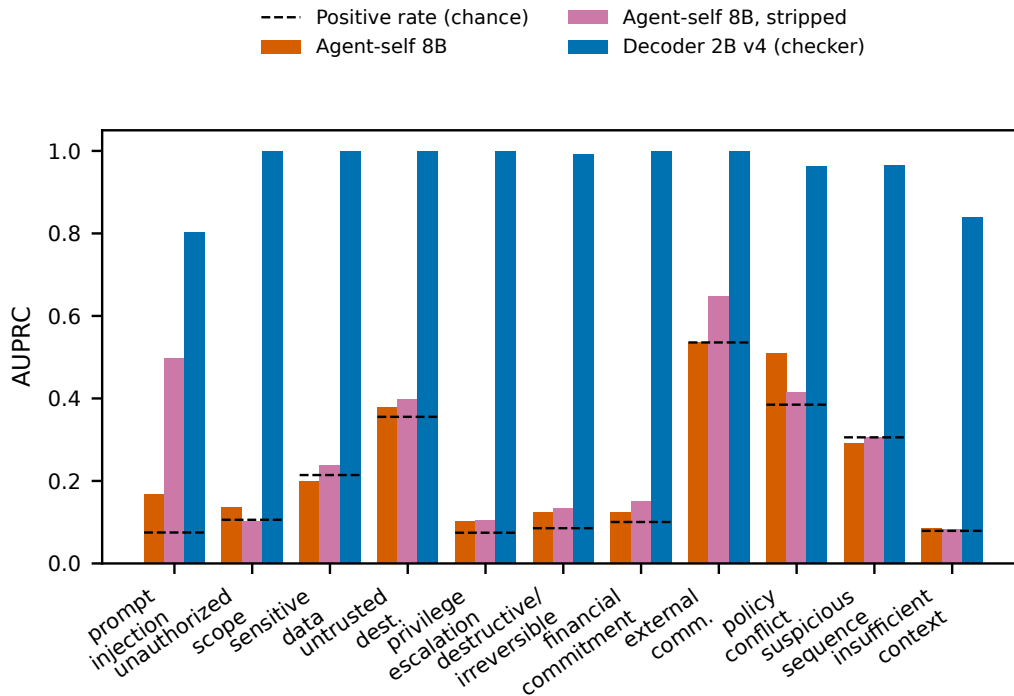


Figure 4: Per-dimension AUPRC on heldout\_family (synthetic data;  $n=2000$  for the agent-self-8B rows,  $n=6653$  for the decoder), with each dimension’s positive rate (chance level) marked by a dashed line. Source: docs/results/synthetic-v2/agent-self-8b/, decoder-2b-v4/heldout\_family-report.md.

information sits in its own prompt and is not turned into a decision. Second, **identity context makes the agent worse, not better, at spotting injection**: stripping the policy and scope text raises `prompt_injection_influence` AUPRC from 0.169 to 0.498 – a long policy preamble appears to dilute attention to an injected instruction elsewhere in the trajectory. Both argue for keeping an external checker in the loop rather than trusting the agent’s own judgment, even on content-style dimensions. The small identity signal that does exist (`unauthorized_scope` AUROC 0.61 full vs. 0.48 stripped) shows the agent reads the context a little; it does not turn that into a usable decision, and the gap to the external checker’s 0.960 macro AUPRC persists regardless.

**Row-for-row comparison on v4** (`agent-self-8b-v4/`). Scoring the decoder 2B v4 checker against the identical 2,000 `heldout_family` rows used for the 8B agent above, it beats the agent’s own single-verdict score on every one of the eleven dimensions, by 0.45–0.90 AUPRC: `unauthorized_scope` 0.999 vs. 0.135, `policy_conflict` 0.964 vs. 0.509, `insufficient_context` 0.841 vs. 0.086, `prompt_injection_influence` 0.804 vs. 0.166 (agent stripped: 0.507). Identity context in the agent’s own prompt still costs it on injection detection even though stripping barely moves the checker; the agent’s own macro is 0.242 full vs. 0.281 stripped.

## 7 ARE THE LABELS REAL? AN INDEPENDENT BLIND JUDGE

Every number so far measures whether an arm learned forecheck’s own generator’s label function (§10). That leaves open a narrower but answerable question: is the label itself observable in the rendered text at all, to a reader who never sees the generator’s latent scenario? We had a frontier model (`gpt-5.6-sol`), given only the rendered context and the eleven dimension definitions, judge 600 v4 rows stratified so every dimension has at least 20 positive and 20 negative examples, across `test`, `heldout_family`, and `heldout_policy_kind`. Agreement with the generator’s label is not a ground-truth check – the judge is another reader of the same text a classifier sees – but low agreement is informative: it means the label depends on information the generator tracked internally and never rendered, so a trained model scoring well on it is learning a generator regularity, not a fact about the text.

Table 6 and Figure 5 split into three tiers. **Recoverable** ( $\kappa > 0.7$ ): `privilege_escalation`, `prompt_injection_influence`, and `policy_conflict`. A blind reader recovers these from the text alone. `privilege_escalation` at 0.91 is an independent confirmation of the v4 render fix (§4): the deciding fields are now genuinely in the tool-call arguments, not just in the model’s training signal. `policy_conflict` at 0.71 with judge recall 0.89 means the policy clause plus the proposed call is enough for an outside reader to spot the conflict; the disagreements are mostly the judge reading a clause more strictly than the generator’s own predicate, not a missing signal.

**Definitional disagreement** ( $0.2 \leq \kappa \leq 0.5$ ): `unauthorized_scope`, `financial_commitment`, `destructive_or_irreversible_action`, `suspicious_action_sequence`, `external_communication`, and `untrusted_destination`. Here the judge and the generator are both reading the text and both producing a coherent decision – they just use different definitions of the same dimension name. For `unauthorized_scope` the judge flags roughly twice as many rows as the generator (159 vs. 81 judge-positive) because it treats a mismatch with the agent’s stated objective (e.g. a bulk retrieval where a single-record lookup was requested) as out of scope, while the generator’s label is entitlement-based only; judge recall of the generator’s positives is still 0.74, so the two notions overlap substantially without being identical. For `financial_commitment` the judge requires funds to actually move, while the generator labels any call that carries a financial amount in its arguments. These are taxonomy decisions to fix in the dimension definitions, not evidence the labels are unobservable.

**Not recoverable** ( $\kappa < 0.1$ ): `insufficient_context` and `sensitive_data_exposure`. For `insufficient_context` the judge calls the context sufficient on 50 of the 58 generator-positive rows in the sample; whatever the generator tracks as a context gap is not visible in what gets rendered. For `sensitive_data_exposure` the judge requires a transfer to a lower-sensitivity destination, while the generator labels any read of confidential data through an already-authorised tool as exposure – recall 0.17, precision 0.08, essentially two different dimensions sharing a name.

Table 6: Independent LLM-judge agreement with the generator label, yes vs. not-yes,  $n=600$  v4 rows (gpt-5.6-sol, blind to the generator’s latent scenario). “Stripped” re-judges the same 600 rows with the `-strip-identity` rendering used for the model identity ablation (§3). Agreement/recall/precision are for the full-context pass. Source: docs/results/judge/README.md.

| Dimension                          | $\kappa$ full | $\kappa$ stripped | Agreement | Judge recall | Judge precision |
|------------------------------------|---------------|-------------------|-----------|--------------|-----------------|
| privilege_escalation               | 0.909         | 0.909             | 0.988     | 1.00         | 0.84            |
| prompt_injection_influence         | 0.905         | 0.905             | 0.983     | 0.84         | 1.00            |
| policy_conflict                    | 0.714         | 0.029             | 0.862     | 0.89         | 0.78            |
| destructive_or_irreversible_action | 0.511         | 0.542             | 0.897     | 0.60         | 0.54            |
| suspicious_action_sequence         | 0.404         | 0.428             | 0.852     | 0.82         | 0.33            |
| unauthorized_scope                 | 0.391         | 0.040             | 0.800     | 0.74         | 0.38            |
| financial_commitment               | 0.359         | 0.300             | 0.935     | 0.24         | 0.92            |
| external_communication             | 0.331         | 0.361             | 0.897     | 0.86         | 0.23            |
| untrusted_destination              | 0.213         | 0.205             | 0.777     | 0.41         | 0.29            |
| insufficient_context               | 0.059         | −0.036            | 0.843     | 0.14         | 0.15            |
| sensitive_data_exposure            | 0.002         | 0.061             | 0.783     | 0.17         | 0.08            |

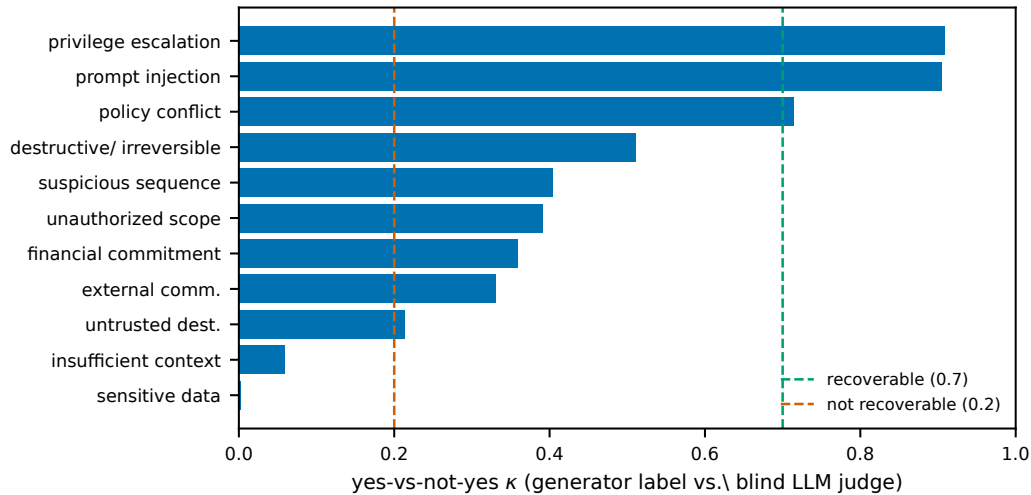


Figure 5: Yes-vs-not-yes  $\kappa$  per dimension, sorted, with the recoverable (0.7) and not-recoverable (0.2) tier lines from §7. Source: docs/results/judge/README.md.

**Consequence for the identity-ablation claim.** §3’s claim that delegated-authority information is load-bearing rests on exactly three dimensions: `unauthorized_scope`, `policy_conflict`, and `insufficient_context`. Judged independently, that claim is solid on `policy_conflict` ( $\kappa=0.71$ ), directionally confirmed but not identical in definition on `unauthorized_scope` ( $\kappa=0.39$ , judge recall 0.74), and unvalidated on `insufficient_context` ( $\kappa=0.06$ ): a blind frontier reader cannot recover this label from the text at all, which means the trained decoder’s 0.849 AUPRC on it (Table 2) is closer to learning a generator regularity than to answering a checkable question about the rendered context. We report `insufficient_context` numbers throughout this paper because the identity-ablation *pattern* – three dimensions collapse together, four content dimensions do not move – still holds with it removed on the other two legs alone; but any claim resting on `insufficient_context` specifically should be read as unconfirmed until the generator makes its context-gap condition explicit in the rendering. The two content dimensions that survive identity stripping in §3 (`prompt_injection_influence`, `privilege_escalation`) are exactly the two the judge recovers best, consistent with them being genuine properties of the text rather than generator lookups.

**Judging without identity context.** We re-ran the same judge on the same 600 rows with identity stripped from the rendered context – the `-strip-identity` rendering used in §3 – to test whether a blind reader loses the same dimensions the trained checker loses. It does, and only those two. `policy_conflict`’s  $\kappa$  falls from 0.714 to 0.029: the judge finds 28 positives instead of 259 because the policy text it would check the action against is no longer in the context. `unauthorized_scope`’s  $\kappa$  falls from 0.391 to 0.040: the judge now flags 203 rows at precision 0.16, which means it is guessing from the action alone rather than reading an entitlement. Every other dimension is unchanged within noise (Table 6); the largest other move is `destructive_or_irreversible_action`, 0.511 to 0.542. `insufficient_context` stays unrecoverable in both passes (0.059 and  $-0.036$ ), the third leg of the identity-ablation claim that remains unvalidated. This is an independent replication of the identity-ablation result on labels rather than on model scores: a blind frontier reader loses the same two dimensions the trained checker loses when identity is stripped (`unauthorized_scope` 0.999 to 0.125 AUPRC, `policy_conflict` 0.964 to 0.392 AUPRC, Table 2), and keeps the same content dimensions (`prompt_injection_influence`, `privilege_escalation`) the checker keeps. The content dimensions’ judge-yes and generator-yes counts are identical between the two passes, which also confirms that stripping identity left the tool-call and trajectory text itself unchanged. One content dimension needs a caveat: the `prompt_injection_influence` agreement ( $\kappa=0.905$ ) was measured on v4 text in which every injected call carried an `instructed_target` argument (§10), so the judge and the generator could agree by reading that key rather than the injected text.

**Scoring the checker against the judge’s labels.** Re-scoring the trained 2B v4 decoder and the rule baseline against the judge’s labels on the judged rows ( $n=257$  on `heldout_family`) gives Table 7. Against labels the generator did not write, the checker keeps the three recoverable dimensions and `untrusted_destination`, and beats the rule baseline on each by 0.08–0.68 AUPRC; macro 0.61 vs. 0.42. `unauthorized_scope` drops to 0.59 for the definitional reason above (the judge counts a mismatch with the stated objective as out of scope; the trained label is entitlement-based). `insufficient_context` at 0.09 confirms the label is a generator artefact. The lookup dimensions collapse for both arms because the judge defines them differently, so they are not evidence for or against either.

Table 7: AUPRC against the independent judge’s labels (gpt-5.6-sol) on the judged heldout\_family rows ( $n=257$ ), decoder 2B v4 vs. rule baseline, with the decoder’s AUPRC against the generator’s own labels on the full split for reference. Source: docs/results/judge/README.md.

| Dimension                          | Decoder 2B v4 | Rule baseline | Decoder vs. generator labels |
|------------------------------------|---------------|---------------|------------------------------|
| prompt_injection_influence         | <b>1.000</b>  | 0.459         | 0.804                        |
| privilege_escalation               | <b>0.977</b>  | 0.297         | 1.000                        |
| policy_conflict                    | <b>0.889</b>  | 0.502         | 0.964                        |
| untrusted_destination              | <b>0.864</b>  | 0.785         | 1.000                        |
| destructive_or_irreversible_action | 0.635         | 0.509         | 0.992                        |
| unauthorized_scope                 | 0.592         | 0.509         | 0.999                        |
| suspicious_action_sequence         | 0.528         | 0.467         | 0.966                        |
| external_communication             | 0.437         | 0.381         | 1.000                        |
| sensitive_data_exposure            | 0.405         | 0.291         | 1.000                        |
| financial_commitment               | 0.327         | 0.280         | 1.000                        |
| insufficient_context               | 0.090         | 0.090         | 0.841                        |
| macro                              | 0.613         | 0.415         | 0.960                        |

## 8 TRANSFER TO AGENTDOJO AND INJECAGENT: DATA WE DID NOT GENERATE

Every result so far is on forecheck’s own synthetic generator (§10). We score the same v4 checkers, zero-shot, on AgentDojo (Debenedetti et al., 2024), a published benchmark of agent run traces we did not write. We use the banking suite traces for claude-3-5-sonnet-20241022 and gpt-4o-2024-05-13: 1,714 runs, 4,215 proposed tool calls, 11 attack types plus no-attack runs. Labels come from AgentDojo’s own `ground_truth()` functions, not from forecheck’s generator: `prompt_injection_influence` is yes when a call executes the injection task’s goal (positive rate 0.150,  $n=4,133$  evaluable calls); `unauthorized_scope` is yes when the call’s function is outside the user task’s reference-solution function set (rate 0.253); `policy_conflict` is yes when one of four banking policies we wrote once per suite fires, checked by deterministic predicates (rate 0.237); `financial_commitment` (0.343) and `destructive_or_irreversible_action` (0.097) come from function-name sets. Trace parsing and the full label derivation are documented in docs/external-agentdojo.md. Every arm is zero-shot: nothing was trained or tuned on AgentDojo, the calibration bundles were fitted on synthetic dev, and threshold selection is disabled.

Table 8 and Figure 6 give six readings. **Policy checking transfers.** 2B v4 scores 0.849 AUPRC on `policy_conflict` against a 0.237 base rate and a rule baseline at base rate, on policies, a tool schema, and agent traces it never saw. Eval-time stripping drops it to 0.694 and the train-time stripped model to 0.390, the same identity dependence found in §3, now on external data. **Injection transfer on v4 was a data effect, not a scale effect.** 2B v4 fails outright (0.128 AUPRC, below the 0.150 base rate, AUROC 0.348); 8B v4 reaches 0.701 AUPRC and 0.895 AUROC zero-shot; the rule baseline sits between the two (0.250, AUROC 0.737) because presence of untrusted content alone is a strong but insufficient cue. A per-example score dump says why 2B v4 fails: its injection head never fires on this domain. Raw logits for all 4,133 evaluable calls lie between  $-5.0$  and  $-3.0$ , the highest-scoring positive is  $-3.25$ , and no call scores above zero; on the same rows the 2B `policy_conflict` head separates cleanly (median raw logit  $+2.5$  for positives,  $-3.5$  for negatives). The 8B v4 head is live but shifted: median raw logit  $-1.75$  for positives against  $-3.50$  for negatives, 29% of positives above zero against 0.7% of negatives. The v6 checkers, trained on the generator with the label leak removed (§10), settle the question: 2B v6 reaches 0.693 AUPRC and 0.928 AUROC, 8B v6 0.838 and 0.966, and both heads now fire (2B v6 median raw  $+4.5$  for positives vs.  $-3.0$  for negatives, 92% vs. 23% above zero; 8B v6  $+6.25$  vs.  $-3.25$ , 94% vs. 8%). The 2B v4 model had learned injection from a surface cue of the synthetic generator that AgentDojo’s injections do not carry; with the cue gone it learned something that transfers. The two 8B baselines separate training from parameter count: the same Granite-3.3-8B base asked ALLOW/STOP as the agent (§6) scores 0.291 AUPRC on injection and Granite Guardian 3.3 8B zero-shot scores 0.103, below base rate. Stripping identity from the input keeps injection (0.743 for 8B

Table 8: AUPRC on AgentDojo banking traces, zero-shot,  $n=4,215$  ( $n=4,133$  for injection). `untrusted_destination` has positives only and is excluded from macro. Guardian and agent-self rows are on a fixed 1,000-row subsample (positive rates 0.155 / 0.269 / 0.246 / 0.357 / 0.091). Source: docs/results/agentdojo/\*-report.md.

| Arm                        | injection | scope | policy | financial | destructive | macro |
|----------------------------|-----------|-------|--------|-----------|-------------|-------|
| Rule baseline              | 0.250     | 0.253 | 0.237  | 0.896     | 1.000       | 0.527 |
| Decoder 2B v4              | 0.128     | 0.257 | 0.849  | 1.000     | 1.000       | 0.647 |
| 2B v4, eval-stripped       | 0.125     | 0.218 | 0.694  | 1.000     | 1.000       | 0.608 |
| 2B v4, train-stripped      | 0.150     | 0.234 | 0.390  | 1.000     | 1.000       | 0.555 |
| Decoder 8B v4              | 0.701     | 0.355 | 0.626  | 0.923     | 0.999       | 0.721 |
| 8B v4, eval-stripped       | 0.743     | 0.266 | 0.404  | 0.934     | 1.000       | 0.669 |
| Decoder 2B v6 (leak-free)  | 0.693     | 0.214 | 0.670  | 0.935     | 1.000       | 0.702 |
| 2B v6, eval-stripped       | 0.669     | 0.306 | 0.521  | 0.941     | 1.000       | 0.687 |
| Decoder 8B v6 (leak-free)  | 0.838     | 0.239 | 0.785  | 0.945     | 0.831       | 0.728 |
| 8B v6, eval-stripped       | 0.819     | 0.354 | 0.450  | 0.979     | 0.975       | 0.715 |
| Granite Guardian 3.3 8B    | 0.103     | 0.289 | 0.224  | 0.257     | 0.217       | 0.218 |
| Agent-self 8B (ALLOW/STOP) | 0.291     | 0.442 | 0.449  | 0.591     | 0.105       | 0.375 |

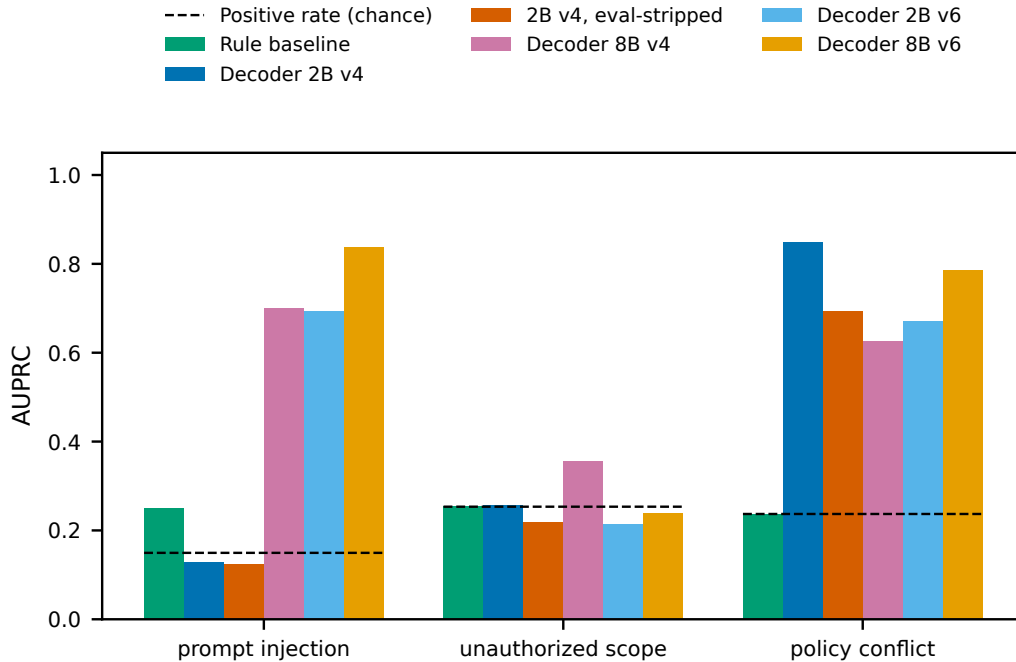


Figure 6: AUPRC on AgentDojo banking traces for the three learned dimensions, rule baseline vs. decoder 2B v4 vs. 2B v4 eval-stripped vs. decoder 8B v4 vs. the leak-free 2B v6 and 8B v6, zero-shot, with each dimension’s positive rate marked by a dashed line. Source: docs/results/agentdojo/\*-report.md.

v4, 0.819 for 8B v6, 0.669 for 2B v6) and drops `policy_conflict` (8B v4 0.626 to 0.404, 8B v6 0.785 to 0.450), the same content versus identity split as in §3; §8.1 tests whether the injection heads read the untrusted text or the call. **unauthorized\_scope does not transfer** (0.257 2B v4, 0.355 8B v4, 0.214 2B v6, 0.239 8B v6, vs. a 0.253 base rate); the 8B v4 head scores above zero on 77% of negatives and 66% of positives, so it fires on nearly every call and carries no signal on this label. The label itself is the weakest one here, since "function not in the reference solution" counts harmless auxiliary reads as out of scope, and we report this without claiming transfer on it. **The 8B-below-2B policy\_conflict gap is not robust.** On v4, 8B scores 0.626 AUPRC against 2B's 0.849 while its AUROC (0.912) stays high; on v6 the order reverses (8B 0.785, 2B 0.670). A second 8B v4 seed on synthetic `heldout_policy_kind` moves `policy_conflict` by 0.03 (0.851 to 0.825), so the v4 gap of 0.22 is larger than seed noise there, but with one seed per (size, data version) cell we can only say the direction depends on the training data; both sizes rank the violations well (AUROC 0.91–0.94) and differ in how many benign calls they place above them. **Calibration does not transfer, an in-domain refit fixes it:** macro ECE runs 0.20–0.27 for the calibrated arms here, against under 0.03 on synthetic data. Refitting per-dimension isotonic calibrators on a random half of the AgentDojo calls and testing on the other half (five seeds) gives macro ECE 0.008 (2B) and 0.010 (8B) with AUPRC unchanged; the 8B `unauthorized_scope` head goes from ECE 0.60 to 0.01 because it learns to output the base rate. Most of the gain is reached with 250–500 labelled calls (`scripts/agentdojo_recalibrate.py`). Last, the labels here are partly written by us (the four policy predicates) and partly by AgentDojo (`prompt_injection_influence`, `unauthorized_scope`).

**Four suites.** Re-running the export over all four AgentDojo suites gives 30,143 proposed calls (banking 4,215, slack 8,246, travel 10,701, workspace 6,981), with injection positive rate 0.099 overall and 0.025 on travel. The 8B v4 checker's injection transfer holds on three of the four: AUPRC/AUROC 0.701/0.895 on banking, 0.752/0.941 on workspace, 0.433/0.738 on slack, each several times the base rate, and 0.131/0.834 on travel where positives are 2.5% of calls; the eval-stripped arm is within 0.04 of it on every suite. 2B v4 is at base rate on every suite; the v6 checkers were not run on the four-suite export. The agent-self baseline reaches 0.251/0.809 over all suites and Guardian 0.116/0.553. `unauthorized_scope` outside banking is carried by the rule baseline (0.749 on slack, 0.733 on workspace, where nearly half of all calls are labelled out of scope); 2B adds under 0.09 and both 8B arms are below the rule. `policy_conflict` on slack, travel, and workspace uses one generic policy per suite at 0.7–8% base rates and is a weak label there: 2B AUROC is 0.78–0.89 but AUPRC 0.005–0.49, so we claim policy transfer on banking only. Per-suite tables are in `docs/results/agentdojo/README.md`.

## 8.1 INJECAGENT: HOLDING THE CALL FIXED

AgentDojo's injection positives differ from their negatives in the call itself: an unrequested transfer to a recipient the user never named. To test whether the checker reads the untrusted text at all, we port InjecAgent (Zhan et al., 2024) (1,054 cases, 38 toolkits, MIT licence). Each case gives the user's own call and, per attacker tool, a *poisoned/clean pair*: the same attacker call, with identical arguments, proposed after the tool response with the attacker's instruction filled in (`prompt_injection_influence=yes`) or left empty (`no`). The only difference inside a pair is the observation (`docs/external-injecagent.md`). 4,250 examples, 1,598 pairs.

**Table 9: neither v4 checker reads the injected text; both v6 checkers do.** With the call held fixed, the 8B v4 injection score rises on 35% of pairs and is unchanged on 39%; both v4 heads sit at raw logit −4 on poisoned and clean alike. This reinterprets the v4 AgentDojo result: what transferred at 8B v4 is call-level, whether the action is something the user asked for, not reading the untrusted content for the instruction that explains it. The v4 generator's leak (§10) is the cause: the training signal was carried by an argument key and the observation never had to be read. Removing it is sufficient. The v6 checkers, identical in size and recipe, rank the poisoned variant above its clean twin on 97.7% of the 1,598 pairs at both sizes (median raw logit +3.5 vs. −6.25 for 2B, +2.25 vs. −3.5 for 8B), reaching 0.878 (2B) and 0.943 (8B) AUPRC on a label whose only evidence is in the untrusted text. **Policy checking transfers a second time:** `policy_conflict` 0.965 (2B) and 0.977 (8B) against a 0.464 base rate on five policies written for this port over 38 unseen toolkits, with the rule baseline at base rate; stripping identity barely moves it here because these five policies are global rules about the call rather than about what the principal named; the v6 checkers hold it (0.922, 0.965). `unauthorized_scope` has signal at 8B v4 (0.849 vs. 0.752

Table 9: AUPRC on InjecAgent, zero-shot, v4 and leak-free v6 checkers,  $n=4,250$  ( $n=3,196$  for injection, positive rate 0.500). Right: fraction of the 1,598 poisoned/clean pairs on which the poisoned variant of the same call scores higher on injection (0.5 is chance). Baseline rows are on a fixed 1,000-row subsample (injection  $n=767$ ). Source: docs/results/injecagent/README.md.

| Arm                        | injection | scope | policy | financial | destructive | macro | pair win |
|----------------------------|-----------|-------|--------|-----------|-------------|-------|----------|
| Rule baseline              | 0.500     | 0.752 | 0.464  | 0.683     | 1.000       | 0.680 | –        |
| Decoder 2B v4              | 0.549     | 0.732 | 0.965  | 1.000     | 1.000       | 0.849 | 0.504    |
| Decoder 8B v4              | 0.544     | 0.849 | 0.977  | 0.734     | 1.000       | 0.821 | 0.353    |
| 8B v4, eval-stripped       | 0.556     | 0.843 | 0.968  | 0.766     | 1.000       | 0.827 | 0.382    |
| Decoder 2B v6 (leak-free)  | 0.878     | 0.704 | 0.922  | 0.970     | 0.998       | 0.894 | 0.977    |
| Decoder 8B v6 (leak-free)  | 0.943     | 0.677 | 0.965  | 0.771     | 0.997       | 0.870 | 0.977    |
| Granite Guardian 3.3 8B    | 0.511     | 0.613 | 0.359  | 0.102     | 0.197       | 0.356 | –        |
| Agent-self 8B (ALLOW/STOP) | 0.530     | 1.000 | 0.647  | 0.061     | 0.178       | 0.483 | –        |

base), none at 2B, and the 8B signal does not survive the v6 retrain (0.677). Both baselines are at chance on injection too (AUROC 0.54 and 0.60); the untrained agent flags every attacker call as STOP (unauthorized\_scope 1.000), the same call-level judgement; the two v6 checkers are the only arms in this table that read the poisoned text.

## 9 RELATED WORK

**LLM guardrails.** Llama Guard (Inan et al., 2023), Granite Guardian (Padhi et al., 2024), Shield-Gemma (Zeng et al., 2024), NVIDIA Aegis (Ghosh et al., 2024), and gpt-oss-safeguard (OpenAI, 2025) are general-purpose content/harm classifiers over a conversation turn or action; Agent Action Guard (Pro-GenAI, 2025) is the same idea at small scale, a sentence-embedding MLP over the tool name and arguments alone (260 labelled rows, one per tool, no identity or policy input). We do not claim they fail at their intended purpose; §3 shows they are near chance specifically on delegated-authority dimensions (unauthorized\_scope, policy\_conflict, insufficient\_context) they were not trained to answer, regardless of whether identity context is in their prompt.

**Agent safety benchmarks.** AgentHarm (Andriushchenko et al., 2024), ToolEmu (Ruan et al., 2023), InjecAgent (Zhan et al., 2024), and AgentDojo (Debenedetti et al., 2024) measure end-to-end agent behavior or injection success via an outcome judge, not principal entitlements or delegated scope as structured input, and produce no per-action calibrated probability a downstream policy engine can compose (§2, §5): they answer "did the agent do something bad," not "should this action have been allowed given who it was for."

**Authorization and policy engines.** OPA (Open Policy Agent), Cedar (Cedar Policy), and XACML (OASIS, 2013) are deterministic, symbolic engines: given a fully specified request and policy, they decide allow/deny with no learned uncertainty. forecheck’s policy layer is architecturally in this family, but the composition problem in §5 exists only because its input is a learned, calibrated probability rather than a fact a symbolic engine evaluates exactly.

**Declared intent and deterministic enforcement.** The OWASP agentic security guidance (OWASP GenAI Security Project, 2025) and practitioner positions built on it (CapiscIO, 2026) hold that an authorization decision must be deterministic and auditable, so a model’s judgment that an action “looks fine” cannot be the control; the agent should declare a typed mandate up front and a policy enforcement point should check each call against it symbolically. Our results support the first half: the acting agent is at chance on its own scope (§6). They also show the limit of the second: once the mandate is a natural-language policy, the symbolic checker (our rule baseline) drops to 0.33–0.50 AUPRC on policy\_conflict and unauthorized\_scope, and only a learned reader recovers it across paraphrases (§4). forecheck keeps the decision deterministic and puts the learned component where OWASP allows it: the scorer never authorises, it prices the review, and §5 measures what that advisory signal is worth.

**Calibration.** Platt/temperature scaling (Platt, 1999), the modern re-examination of network calibration (Guo et al., 2017), and the Wilson score interval (Wilson, 1927) (used for the approval-elimination bound, Appendix A) are standard tools we apply directly to policy *composition*, not where this literature is usually pointed: §5’s  $k$ -invariance result is a direct function of the underlying model’s calibration sharpness.

## 10 LIMITATIONS

**Synthetic data only.** Every number here is on data from forecheck’s own scenario generator (docs/evaluation-plan.md, classes 1–2). No human-labelled or production-traffic evaluation exists at submission time, and no real-world safety claim is made: every result measures whether an arm learned the generator’s own label function.

**Independent labels exist for label validity, not for real-world risk.** §7 adds a blind LLM judge as an independent reader of the same rendered text, which is a check on whether a label is observable in the text at all, not a check against real-world ground truth – the judge never saw production traffic either. That check is reassuring on `policy_conflict` ( $\kappa=0.71$ ) and directionally consistent on `unauthorized_scope` ( $\kappa=0.39$ , differing definitions), but finds `insufficient_context` ( $\kappa=0.06$ ) unrecoverable from the text by an independent reader. Every `insufficient_context` number in this paper should be read with that caveat attached until the generator’s rendering makes the context gap explicit.

**Generator regularities.** The model may be exploiting template regularities in how the generator writes policy clauses and entitlement lists, rather than a general policy-reading ability. The `heldout_policy_kind/heldout_policy_phrasing` splits (§4) probe this and only partly rule it out: phrasing generalizes cleanly, kind does not, consistent with a lookup over kinds rather than the underlying operation. The encoder’s earlier null result on both splits is uninformative the same way – it never learned `policy_conflict` at all, so a flat curve shows nothing.

**`privilege_escalation` render defect (v2/v3).** The `authority_before/authority_after` fields deciding this label were never rendered into the tool-call text in v2/v3, so every arm’s 0.46–0.53 AUPRC there reflects a data defect: 78 of 79 dev positives were textually indistinguishable from an in-scope re-grant. Fixed in v4 (§4); numbers sourced from v2/v3 elsewhere should be read with this in mind.

**`prompt_injection_influence` label leak (v1–v5).** After the 2B checker failed on AgentDojo injection (§8) we contrasted the rendered call between injected and non-injected synthetic examples. The generator wrote an `instructed_target` argument into every injected call and into no other call (presence 1.00 vs. 0.00 over 13,312 v4 rows), put a `to` argument into 100% of injected calls vs. 3% of others, and drew the injected target from five literals, three containing the word “attacker”. A key-presence detector would score 1.0 AUPRC on this dimension, above every trained arm’s  $\sim 0.80$ , so the synthetic injection numbers in this paper (Tables 2, 7) and the judge’s  $\kappa=0.905$  on it are not evidence of injection detection; the judge saw the same argument. The identity dimensions do not use the leaked keys and are unaffected. The AgentDojo and InjecAgent results are the first leak-free measurements of the v4 checkers: 2B fails (0.128 AUPRC), 8B partly transfers (0.701). The generator is fixed in v6 (renderer 2.0.0: no `instructed_target`, target argument key chosen from the tool kind and filled for benign calls too, shared target pools, 300–2,000 character multi-format observations) with a leakage check on argument-key presence parity. Retrained on v6, the synthetic injection AUPRC is unchanged (0.802 to 0.788 for 2B, 0.791 to 0.795 for 8B) while the AgentDojo injection AUPRC rises to 0.693 (2B) and 0.838 (8B) and the InjecAgent poisoned-over-clean pair win rises from 0.504 (2B) and 0.353 (8B) to 0.977 at both sizes (§8): the leak did not inflate the synthetic score, it changed what the head learned to read (docs/results/notes/injection-label-leak-diagnosis.md). Every other synthetic dimension is within 0.01 of v4 except `policy_conflict` on `heldout_policy_kind`, which drops to 0.692 (2B) and 0.767 (8B), inside the seed band below for 2B and 0.08 below the single v4 seed for 8B.

**Seed variance (three seeds each for v4 and v5).** Three training seeds for decoder v4 agree on macro AUPRC to within 0.001 on every split (e.g. 0.960, 0.959, 0.960 on test). But the `heldout_policy_kind` `policy_conflict` cell moves across a 0.06 range between seeds

(0.752, 0.689, 0.711; mean 0.717, sample sd 0.032) – an order of magnitude more than the macro noise floor – so differences smaller than that on this specific cell are not distinguishable from seed noise. Three seeds of the v5 (30-kind) recipe show the same range (0.704, 0.755, 0.717; mean 0.725, sample sd 0.026), which is what lets §4 compare the two bands directly instead of a single point each. The 8B numbers are from one seed except where noted; a second 8B seed reaches macro AUPRC 0.948 on test against 0.956 for the first, with the whole gap on `insufficient_context` (0.682 vs. 0.767) and the other ten dimensions within 0.003, so 8B-vs-2B gaps on that dimension are inside seed noise. On `heldout_policy_kind` the second seed puts `policy_conflict` at 0.825 against 0.851, inside the 2B seed band above.

**Held-out-policy-kind failure (now tested at 30 kinds and 8B).** `policy_conflict` AUPRC drops from  $\sim 0.98$ – $0.99$  (in-distribution, unseen-phrasing) to 0.65–0.76 on entirely unseen kinds at 2B. The v5 experiment (ADR 0011, 30 kinds, 4 withheld) has landed across three seeds and is negative: mean 0.725, inside one sample sd of v4’s mean 0.717 – tripling the kind inventory did not fix this at 2B. An 8B LoRA on the unchanged v4 data reaches 0.851, so capacity helps partially but does not solve it. The supportable claim today is "one model, policies of a kind seen in training (or with enough capacity to partially generalize), any phrasing" – not "arbitrary new policy kinds."

**External labels are partly self-written and uneven across suites.** The AgentDojo labels (§8) are partly written by us (the policy predicates) and partly by AgentDojo (`prompt_injection_influence`, `unauthorized_scope`). The banking suite has four tool-specific policies; slack, travel, and workspace have one generic policy each, so the policy-transfer claim rests on banking alone, and the four-suite injection result is one training run (8B v4, with and without identity) on a label whose positive rate is 2.5% on travel. Calibration did not transfer: macro ECE runs 0.20–0.27 on AgentDojo against under 0.03 on synthetic data, so only the ranking generalizes to this external distribution. Refitting isotonic calibrators on half of the AgentDojo calls brings macro ECE on the other half to 0.008 (2B) and 0.010 (8B) with AUPRC unchanged, and most of the gain arrives by 250–500 labelled calls (`docs/results/agentdojo/recalibration.md`), so a deployment needs a few hundred in-domain labels for its probabilities, not for its ranking.

**Other caveats.** A single generator family (English, template renderer) produces every example. The §5 cost table is the default severity tier, not tuned to any organization. Generator seed nondeterminism was found and partially fixed mid-project (`docs/results/notes/privilege-escalation-diagnosis.md`); v2/v3 data are not byte-reproducible from their configs.

## 11 CONCLUSION

Pre-execution risk for agent tool calls is mostly a delegated-authority problem, not a content-safety problem, and general-purpose safety models do not solve it: stripping identity and policy context from a trained classifier’s input collapses exactly the dimensions that depend on it, and two zero-shot safety models are at chance on those same dimensions whether or not the context is available to them. Identity/delegation context and calibration quality are not incidental engineering details for agent guardrails – they are the two properties that determine whether a guardrail can detect authorization violations at all (§3) and survive being deployed alongside more than one policy (§5). What remains open is whether policy-conditioned reasoning generalizes to genuinely novel policy kinds, not just novel wording of familiar ones (§4); the v5 experiment testing this, and a first real-data probe against hand-labelled traffic, are the next things this line of work needs before any claim above can be extended past synthetic data. The AgentDojo and InjecAgent results (§8) are a first step on that last point: the policy-conflict ranking holds zero-shot on agent traces and policies forecheck never saw, the injection ranking holds only where the call itself carries the evidence, and the probabilities need a few hundred in-domain labels to refit. The external evaluation also exposed a label leak in our generator that the synthetic splits could not, and fixing it turned the 2B checker’s injection failure on AgentDojo into a 0.69 AUPRC transfer and its chance-level InjecAgent pair discrimination into 0.98 at both sizes, while the synthetic score on the same dimension did not move. The case for evaluating on data one did not generate is the strongest result in this paper.

---

## REFERENCES

- Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Eric Winsor, Eliot Wynne, Yarin Gal, and Xander Davies. Agentharm: A benchmark for measuring harmfulness of llm agents. *arXiv preprint arXiv:2410.09024*, 2024.
- CapiscIO. Automated intent classification for AI agents is an unsolved problem. <https://capisc.io/blog/automated-intent-classification-for-ai-agents-is-an-unsolved-problem>, 2026. Vendor position piece. Accessed 2026-09-22.
- Cedar Policy. Cedar policy language documentation. <https://www.cedarpolicy.com/en/documentation>. Accessed 2026-09-22.
- Edoardo DeBenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *arXiv preprint arXiv:2406.13352*, 2024.
- Shaona Ghosh, Prasoon Varshney, Erick Galinkin, and Christopher Parisien. Aegis: Online adaptive ai content safety moderation with ensemble of llm experts. *arXiv preprint arXiv:2404.05993*, 2024.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- IBM Granite Team. Granite 3.0 / 3.3 language models: Model card and technical report. Technical report, IBM Research, 2024. See also Granite 3.3 (2025) release notes.
- Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabza. Llama guard: Llm-based input-output safeguard for human-ai conversations. *arXiv preprint arXiv:2312.06674*, 2023.
- OASIS. extensible access control markup language (xacml) version 3.0. OASIS Standard, <http://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html>, 2013.
- Open Policy Agent. Open policy agent (opa) documentation. <https://www.openpolicyagent.org/docs/latest/>. Accessed 2026-09-22.
- OpenAI. gpt-oss-safeguard: Open-weight reasoning models for safety policy classification. Technical report, OpenAI, 2025. Model and technical report.
- OWASP GenAI Security Project. OWASP securing agentic applications guide. <https://genai.owasp.org/>, 2025. Accessed 2026-09-22.
- Inkit Padhi, Manish Nagireddy, Giandomenico Cornacchia, Subhajit Chaudhury, Tejaswini Pedapati, Pierre Dognin, Keerthiram Murugesan, Erik Miehl, Martin Santillan Qian, Ramya Ramamurthy, Giulio Zizzo, Ambrish Eltokhy, Michael Hind, Prasanna Ram, Payel Das, Pin-Yu Chen, Jerret Cunningham, Ananya Agarwal, Prasanna Sattigeri, Kush Raghavan, and Kush R. Varshney. Granite guardian. *arXiv preprint arXiv:2412.07724*, 2024.
- John Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, pp. 61–74, 1999.
- Pro-GenAI. Agent action guard: Classifying AI agent actions to ensure safety and reliability. Preprint, <https://doi.org/10.20944/preprints202510.1415.v2>; code <https://github.com/Pro-GenAI/Agent-Action-Guard>, 2025. Accessed 2026-09-22.

Table 10: Multi-policy stacking, heldout\_family ( $n=6640$ ). Source: docs/results/synthetic-v2/README.md, docs/evaluation/multi-policy-stacking.md.

| Arm                | $k$ | Strategy            | FPR          | FNR   | Review rate |
|--------------------|-----|---------------------|--------------|-------|-------------|
| Decoder 2B         | 1   | any                 | 0.015        | 0.253 | 0.31        |
| Decoder 2B         | 2   | independent         | 0.140        | 0.080 | 0.30        |
| Decoder 2B         | 2   | expected_cost_joint | 0.082        | 0.070 | 0.28        |
| Decoder 2B         | 3   | independent         | <b>0.966</b> | 0.016 | 0.66        |
| Decoder 2B         | 3   | joint               | 0.905        | 0.051 | 0.63        |
| Decoder 2B         | 3   | expected_cost_joint | 0.082        | 0.070 | 0.28        |
| Decoder 2B         | 8   | independent         | 0.966        | 0.007 | 0.58        |
| Decoder 2B         | 8   | expected_cost_joint | 0.082        | 0.070 | 0.28        |
| Encoder granite-r2 | 2   | independent         | 0.710        | 0.042 | 0.26        |
| Encoder granite-r2 | 2   | expected_cost_joint | 0.921        | 0.010 | 0.37        |
| Encoder granite-r2 | 3   | independent         | 0.990        | 0.008 | 0.21        |
| Encoder granite-r2 | 3   | expected_cost_joint | 0.921        | 0.010 | 0.37        |

Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of lm agents with an lm-emulated sandbox. *arXiv preprint arXiv:2309.15817*, 2023.

Benjamin Warner, Antoine Chaffin, Benoît Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Nathan Cooper, Griffin Adams, Jeremy Howard, and Iacopo Poli. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. *arXiv preprint arXiv:2412.13663*, 2024.

Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

Wenjun Zeng, Yuchen Liu, Ryan Mullins, Ludovic Peran, Joe Fernandez, Hamza Harkous, Karthik Narasimhan, Drew Proud, Piyush Kumar, Bhaktipriya Radharapu, Olivia Sturman, and Oscar Wahltinez. Shieldgemma: Generative ai content moderation based on gemma. *arXiv preprint arXiv:2407.21772*, 2024.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. *arXiv preprint arXiv:2403.02691*, 2024.

## A APPENDIX

### A.1 FULL MULTI-POLICY STACKING TABLE

Table 10 extends Table 4 with the `joint` strategy (rules from all  $k$  policies merged into one bundle and evaluated once in threshold mode, rather than decided independently and OR-ed), false negative rate, review rate, and the `granite-embedding-r2` encoder run on the identical examples and synthetic per-dimension policies. `independent` and `joint` are mathematically identical at every  $k$  here because the synthetic stacking policies use only plain, non-hard, non-allow\_override rules, so there is nothing for the engine’s combination logic to do differently between the two strategies on this construction (docs/evaluation/multi-policy-stacking.md §3); the two rows diverge only once real bundles with `hard/allow_override` rules are mixed in, which is outside this diagnostic.

Note the encoder’s `expected_cost_joint` FPR (0.921) is *worse* than its own `independent` FPR at  $k=2$  (0.710): on a poorly calibrated probability vector, aggregating across dimensions does not reliably help and can hurt, underscoring that the composition benefit in §5 is conditional on calibration quality, not a property of the `expected_cost` rule in isolation.

Table 11: Decoder 2B, approval elimination on heldout\_family (v2  $n=6640$ , v4  $n=6653$ ), bundle balanced. “raw” is the split’s own 57–58% base incident rate; “rw” (reweighted) importance-reweights rows so the weighted incident rate equals 5%, matching a plausible deployment rate. Source: docs/results/synthetic-v2/README.md, docs/evaluation/approval-elimination.md.

| Budget | Elim. (v2, raw) | Elim. (v2, rw) | Elim. (v4, rw) | Review (v4, rw) |
|--------|-----------------|----------------|----------------|-----------------|
| 0.1%   | 0.000           | 0.000          | 0.000          | 0.969           |
| 0.5%   | 0.000           | 0.000          | 0.000          | 0.969           |
| 1%     | 0.000           | 0.888          | 0.903          | 0.067           |
| 2%     | 0.000           | 0.961          | 0.952          | 0.021           |
| 5%     | 0.000           | 0.992          | 0.992          | 0.001           |

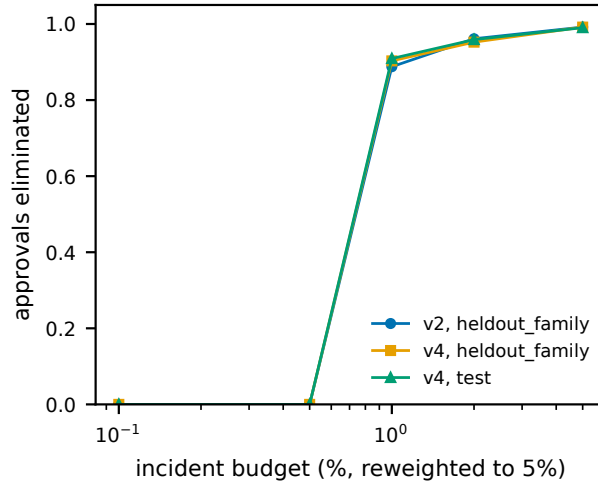


Figure 7: Approvals eliminated vs. incident budget, reweighted to a 5% deployment incident rate, for decoder 2B v2 (heldout\_family,  $n=6640$ ) and v4 (heldout\_family  $n=6653$ , test  $n=4412$ ), all synthetic data, log-scaled budget axis. Source: docs/results/synthetic-v2/decoder-2b/heldout\_family-approval-report.md, decoder-2b-v4/{heldout\_family,test}-approval-report.md.

## A.2 APPROVAL ELIMINATION

The product-facing metric is not classifier accuracy but how many of a deployment’s human-approval reviews can be safely automated at a fixed, chosen incident rate: sort examples by the decoder’s expected cost of ALLOW, and ask what fraction of the review queue can be auto-decided while the fraction of auto-allowed actions that are actually risky stays under a budget. Because the evaluated heldout\_family split is adversarial-heavy (58% base incident rate), Table 11 reports both the raw unweighted curve and a curve importance-reweighted to a stated 5% deployment incident rate, with elimination at each budget computed conservatively via a 95% Wilson score upper bound (Wilson, 1927) on the realized incident rate in the auto-allowed set.

Figure 7 shows the three curves nearly overlapping: v2 and v4 eliminate a comparable share of approvals at every budget on heldout\_family, and v4’s test split tracks its own heldout\_family curve closely, so the elimination curve is not sensitive to the v3/v4 generator fix or to family-held-out tools.

At a deployment incident rate of 5%, the calibrated v2 decoder auto-allows 88.8–99.2% of actions that would otherwise need human review while keeping the allowed-but-risky rate at or below the stated budget. The unweighted column is near zero not because the model is uninformative but because the split’s own top-ranked rows already contain risky examples by construction (an adversarial benchmark, not a deployment population); it is a ceiling of the split’s design, not of the classifier.

---

1080 Below a 1% budget the Wilson bound cannot be met at this split's effective sample size and the  
1081 conservative curve reports zero elimination; a point-estimate mode is available for that regime but is  
1082 dominated by single rows at small allow-set sizes.  
1083

### 1084 A.3 EXPERIMENT MATRIX 1085

1086 The full list of experiments this paper still needs, with recipe files and status, is tracked in  
1087 `paper/EXPERIMENTS.md` rather than duplicated here.  
1088  
1089  
1090  
1091  
1092  
1093  
1094  
1095  
1096  
1097  
1098  
1099  
1100  
1101  
1102  
1103  
1104  
1105  
1106  
1107  
1108  
1109  
1110  
1111  
1112  
1113  
1114  
1115  
1116  
1117  
1118  
1119  
1120  
1121  
1122  
1123  
1124  
1125  
1126  
1127  
1128  
1129  
1130  
1131  
1132  
1133